

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №7
по дисциплине: Компьютерные сети
тема: «Протоколы SMTP и POP3»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей
Александрович

Проверил:
Рубцов Константин Анатольевич

Вариант 2.

Цель работы: изучить принципы и характеристику протоколов POP3 и SMTP и составить программу для приема/отправки электронной почты.

Теоретические сведения Протокол POP3

POP3 (англ. Post Office Protocol Version 3) - стандартный Интернет-протокол прикладного уровня, используемый клиентами электронной почты для извлечения электронного сообщения с удаленного сервера по TCP/IP-соединению [22].

В некоторых небольших узлах Интернет бывает непрактично поддерживать систему передачи сообщений (MTS - Message Transport System). Рабочая станция может не иметь достаточных ресурсов для обеспечения непрерывной работы SMTP-сервера [RFC-821]. Для “домашних ЭВМ” слишком дорого поддерживать связь с Интернет круглые сутки.

Но доступ к электронной почте необходим как для таких малых узлов, так и индивидуальных ЭВМ. Для решения этой проблемы разработан протокол POP3 (Post Office Protocol - Version 3, STD- 53. M. Rose, RFC-1939). Этот протокол обеспечивает доступ узла к базовому почтовому серверу.

POP3 не ставит целью предоставление широкого списка манипуляций с почтой. Почтовые сообщения принимаются почтовым сервером и сохраняются там, пока на рабочей станции клиента не будет запущено приложение POP3. Это приложение устанавливает соединение с сервером и забирает сообщения оттуда. Почтовые сообщения на сервере стираются.

POP3 поддерживает простые требования «загрузи-и-удали» для доступа к удаленным почтовым ящикам. Хотя большая часть POP- клиентов предоставляют возможность оставить почту на сервере после загрузки, использующие POP клиенты обычно соединяются, извлекают все письма, сохраняют их на пользовательском компьютере как новые сообщения, удаляют их с сервера, после чего разъединяются.

Другие протоколы, в частности IMAP, предоставляют более полный и комплексный удаленный доступ к типичным операциям с почтовым ящиком. Многие клиенты электронной почты поддерживают как POP, так и IMAP; однако, гораздо меньше интернет-провайдеров поддерживают IMAP.

POP3-сервер прослушивает порт 110. Шифрование связи для POP3 запрашивается после запуска протокола, с помощью либо команды STLS (если она поддерживается), либо POP3S, которая соединяется с сервером используя TLS или SSL по TCP-порту 995.

Доступные сообщения клиента фиксируются при открытии почтового ящика POP-сессией и определяются количеством сообщений для сессии, или, по желанию, с помощью уникального идентификатора, присваиваемого сообщению POP-сервером. Этот уникальный идентификатор является постоянным и уникальным для почтового ящика и позволяет клиенту получить доступ к одному и тому же сообщению в разных POP-сессиях. Почта извлекается и помечается для удаления с помощью номера сообщения. При выходе клиента из сессии помеченные сообщения

удаляются из почтового ящика.

Обычно POP3-сервис устанавливается на 110-й TCP -порт сервера, который будет находиться в режиме ожидания входящего соединения. Когда клиент хочет воспользоваться POP3-сервисом, он просто устанавливает TCP-соединение с портом 110 этого хоста. После установления соединения сервис POP3 отправляет подсоединившемуся клиенту приветственное сообщение. После этого клиент и сервер начинают обмен командами и данными. По окончании обмена POP3-канал закрывается.

Команды POP3 состоят из ключевых слов, состоящих из ASCII-символов, и одним или несколькими параметрами, отделяемыми друг от друга символом "пробела" - <SP>. Все команды заканчиваются символами "возврата каретки" и "перевода строки" - <CRLF>. Длина ключевых слов не превышает четырех символов, а каждого из аргументов может быть до 40 символов.

Ответы POP3-сервера на команды состоят из строки статус-индикатора, ключевого слова, строки дополнительной информации и символов завершения строки. Длина строки ответа может достигать 512 символов. Строка статус-индикатора принимает два значения: положительное ("OK") и отрицательное ("-ERR"). Любой сервер POP3 обязан отправлять строки статус-индикатора в верхнем регистре, тогда как другие команды и данные могут приниматься или отправляться как в нижнем, так и в верхнем регистрах. Ответы POP3-сервера на отдельные команды могут составлять несколько строк. В этом случае строки разделены символами <CRLF>. Последнюю строку информационной группы завершает строка, состоящая из символа "." (код — 046) и <CRLF>, т. е. последовательность "CRLF.CRLF".

POP3-сессия состоит из нескольких частей. Как только открывается TCP-соединение и POP3-сервер отправляет приветствие, сессия должна быть зарегистрирована - состояние аутентификации (AUTHORIZATION state). Клиент должен зарегистрироваться в POP3- сервере, т. е. ввести свой идентификатор и пароль. После этого сервер предоставляет клиенту его почтовый ящик и открывает для данного клиента транзакцию - состояние начала транзакции обмена (TRANSACTION state). На этой стадии клиент может считать и удалить почту своего почтового ящика.

После того как клиент заканчивает работу (передает команду QUIT), сессия переходит в состояние UPDATE - завершение транзакции. В этом состоянии POP3-сервер закрывает транзакцию данного клиента (на языке баз данных - операция COMMIT) и закрывает TCP-соединение. В случае получения неизвестной, неиспользуемой или неправильной команды, POP3-сервер должен ответить отрицательным состоянием индикатора. POP3-сервер может использовать в своей работе таймер контроля времени соединения. Этот таймер отсчитывает время "бездействия" ("idle") клиента в сессии от последней переданной команды. Если время сессии истекло, сервер закрывает TCP-соединение, не переходя в состояние UPDATE (иными словами, откатывает транзакцию или на языке баз данных — выполняет ROLLBACK) [25].

POP3-сервер может обслуживать группу клиентов, которые, возможно, присоединяются по коммутируемой линии, и, следовательно, необходимо иметь средство автоматического регулирования времени соединения. По спецификации POP3-таймер контроля состояния "idle" должен быть установлен на промежуток времени не менее 10 минут.

Команды протокола POP3:

- USER - идентифицирует пользователя с указанным именем;
- PASS - указывает пароль для пары клиент-сервер;
- QUIT - закрывает TCP-соединение;
- STAT - сервер возвращает количество сообщений в почтовом ящике плюс размер почтового ящика;
- LIST - сервер возвращает идентификаторы сообщений вместе с размерами сообщений;
- RETR - извлекает сообщение из почтового ящика;
- DELE - отмечает сообщение для удаления;
- NOOP - Сервер возвращает положительный ответ, но не совершает никаких действий;
- LAST - Сервер возвращает наибольший номер сообщения из тех, к которым ранее уже обращались;
- RSET - Отменяет удаление сообщения, отмеченного ранее командой DELE.

Протокол SMTP

SMTP (Simple Mail Transfer Protocol) - широко используемый сетевой протокол, предназначенный для передачи электронной почты в сетях TCP/IP. SMTP впервые был описан в RFC 821 (1982 год); последнее обновление в RFC 5321 (2008) включает масштабируемое расширение - ESMTP (Extended SMTP). В настоящее время под «протоколом SMTP», как правило, подразумевают и его расширения. Протокол SMTP предназначен для передачи исходящей почты, используя для этого порт TCP 25 [23].

Упрощенно схема взаимодействия представлена на рис. 7.1 (объемными стрелками показано направление движения почтовых сообщений).

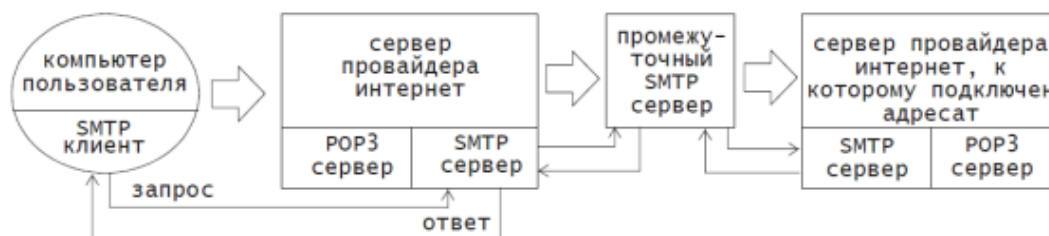


Рис. 7.1. Отправка почты по протоколу SMTP

Со стороны пользователя обычно одна и та же программа выступает в роли и POP3 клиента, и SMTP клиента отправителя. Наиболее распространенными на данный момент являются MS Outlook, The Bat, Netscape Messenger, Eudora, Pegasus mail, Mutt, Pine и др. При нажатии в них на кнопку "отправить" происходит формирование очереди сообщений, и установление двустороннего сеанса общения с SMTP сервером провайдера. На схеме у пользователя есть клиентское ПО, а у провайдера – серверная часть приложения. На самом деле это немного не так. Протокол SMTP делает возможным смену сторон даже в ходе одного сеанса. Условно принято считать клиентом ту сторону, которая начинает взаимодействие и хочет отослать почту, а сервером ту, что принимает запросы. После того, как клиент посылает серверу несколько служебных команд и получает положительные ответы на них, он отправляет SMTP серверу собственно тело сообщения. SMTP сервер получает сообщение, вносит в него дополнительные заголовки, указывающие на то, что он обработал данное послание, устанавливает связь со следующим SMTP сервером по пути следования письма. Общение между любыми SMTP серверами происходит по той же схеме. Иницирует переговоры клиент, сервер на них отвечает, а затем получает корреспонденцию и "ставит штампик" в теле письма (в его заголовочной части). Все это очень напоминает обычную бумажную почту, где работу по сортировке и отправке почты выполняют люди.

Если на каком-нибудь этапе передачи SMTP клиент обнаружит невозможность подключиться к следующему серверу (например, компьютер отправили на профилактику или аппаратура связи вышла из строя), он будет пытаться отправить сообщение через некоторое время – 1 час, 4 часа, день и т.д. до 4 суток в общем случае. Причем временные отрезки между попытками, как правило, зависят от настроек программы-пересыльщика почты. Одновременно, такой сервер должен уведомить отправителя сообщения о невозможности доставить почту, послав ему стандартное письмо "Failed delivery" (доставка невозможна) и рассказав о графике дальнейших попыток по продвижению исходного сообщения. Если канал связи не восстановится за указанный большой промежуток времени (например, 4 дня), посланная информация будет считаться утерянной.

Как только почта достигнет конечного пункта (SMTP сервера адресата сообщения), она будет сложена в почтовый ящик абонента, который всегда сможет в удобное для него время изъять ее по протоколам POP3 или IMAP, в зависимости от того, какой из них поддерживается провайдером.

Анализируя заголовок письма, можно узнать какими путями оно путешествовало, как долго длился сам путь, как называлась почтовая программа отправителя и многое другое. Получить эту информацию можно в "Свойствах письма", кликнув правой кнопкой мыши на самом письме в MS Outlook, нажав Ctrl+Shift+N в The Bat или совершить нечто подобное в других почтовых клиентах.

Рассмотрим клиент-серверное взаимодействие по протоколу SMTP. Программа пользователя, выбрав для связи соответствующий почтовый сервер, устанавливает с ним контакт на транспортном и сеансовом уровнях эталонной модели взаимодействия открытых систем OSI/RM (в терминах

TCP/IP (transmission control protocol / internet protocol) это – TCP уровень). Взаимодействие на более низких уровнях (канальном, сетевом) происходит прозрачно для обеих сторон. Протокол SMTP – протокол прикладного уровня и базируется поверх TCP. В его рамках не оговаривается ни размер сегментов данных, ни правила квитирования, ни отслеживание ошибок, возникающих при передаче информации.

По уже установленному соединению клиентское ПО передает команды SMTP серверу, ожидая тут же получить ответы. В арсенал SMTP клиента, равно как и сервера, входит около 10 команд, но, воспользовавшись только пятью из них, уже можно легально послать почтовое сообщение. Это HELO, MAIL, RCPT, DATA, QUIT. Их использование подразумевается именно в такой последовательности. HELO предназначена для идентификации отправителя, MAIL указывает адрес отправителя, RCPT – адрес назначения. После команды DATA и ответа на нее, клиент посылает серверу тело сообщения, которое должно заканчиваться строкой, содержащей лишь одну точку.

Непосредственно после установления соединения сервер выдает строчку с кодом ответа 220. В ответ на нее клиент может инициировать сеанс связи по протоколу SMTP, послав команду HELO и указав у нее в аргументах имя своего компьютера. По принятии команды HELO сервер обязан сделать запрос в DNS и, если это возможно, по IP адресу определить доменное имя компьютера клиента. (IP адрес уже известен на момент установления соединения по TCP протоколу).

Далее в команде "MAIL FROM:" клиент сообщает обратный адрес отправителя, который проверяется обычно только на корректность. После слов "RCPT TO:" следует набрать адрес электронной почты абонента на данном сервере. Клиент отправляет команду DATA и ждет приглашения начать пересылку тела письма (код 354).

Сообщение может быть достаточно длинным, но обязательно должно заканчиваться строкой, в которой есть одна-единственная точка. Это служит сигналом SMTP серверу о том, что тело письма закончилось. Он присваивает этому письму определенный идентификатор, и ждет команды QUIT, после чего сеанс считается завершенным.

Если клиент посылает сообщение, у которого в заголовочной части в поле CC указаны несколько e-mail адресов, первый по пути следования SMTP сервер должен будет в общем случае установить сеанс продвижения почты с каждым из серверов данного списка и отослать точную копию письма каждому. В случае использования поля BCC клиент, формирующий сообщение, уничтожит запись BCC в теле сообщения и по количеству адресатов отошлет первому SMTP серверу команду "RCPT TO:" каждый раз с новым адресом в качестве аргумента. Таким образом, сервер получит указание разослать почту по многим адресатам. При этом, в этом случае получатели писем ничего не будут знать друг о друге, т.к. рассылка осуществляется посредством команд SMTP протокола.

Анализ применяемых функций

Для использования библиотеки применяется WSAStartup, WSACleanup и WSAGetLastError для отслеживания ошибок и логирования.

Для реализации подключения использованы функции socket и connect. На вход подавались адреса серверов SMTP и POP3, с соответствующими незащищёнными портами 25 и 110.

Для отправки команд используется send. Для получения ответа от сервера используется recv если это небольшие сообщения и recv в цикле если приходит множество строк.

Листинги исходного кода:

```
#include <winsock2.h>
#include <ws2tcpip.h>
#include <stdio.h>
#include <iostream>
#include <string>
#include <windows.h>
#include <schannel.h>
#include <limits>

#define STR_BUF_SIZE 256
#define TEXT_BUF_SIZE 256
#define SMTP_PORT 25
#define POP3_PORT 110

using namespace std;

void throw_error_with_code()
{
    std::string error_msg = "Error with code: ";
    error_msg += to_string(WSAGetLastError());
    throw std::runtime_error(error_msg);
}

const char *getaddrbyname(const char *hostname)
{
    struct hostent *host_info;
    struct in_addr addr = {0};
    char *ip = NULL;

    if ((host_info = gethostbyname(hostname)) == NULL)
        return NULL;

    addr.s_addr = *(u_long *)host_info->h_addr_list[0];

    return inet_ntoa(addr);
}

void startup_wsa();
SOCKET get_socket_descriptor();
sockaddr_in get_bind_addr(const char *address, unsigned short port);
void connect_socket_with_address(SOCKET socket_descriptor, sockaddr_in bind_addr);

SOCKET get_connect(const char *address, unsigned short port)
{
    std::clog << "start connect..." << std::endl;

    startup_wsa();

    std::clog << "WSA started..." << std::endl;

    SOCKET socket_descriptor = get_socket_descriptor();
```



```

        std::clog << "create socket" << std::endl;

        sockaddr_in bind_addr = get_bind_addr(address, port);
        std::clog << "create bind address" << std::endl;

        connect_socket_with_address(socket_descriptor, bind_addr);
        std::clog << "connect socket with address\nconnected" << std::endl;

        return socket_descriptor;
    }

void startup_wsa()
{
    WORD wVersionRequested;
    WSADATA wsaData;
    wVersionRequested = MAKEWORD(2, 2);

    if (WSAStartup(wVersionRequested, &wsaData) == SOCKET_ERROR)
        throw_error_with_code();
}

SOCKET get_socket_descriptor()
{
    SOCKET res = socket(
        AF_INET,
        SOCK_STREAM,
        0);

    if (res == INVALID_SOCKET)
        throw_error_with_code();

    return res;
}

sockaddr_in get_bind_addr(const char *address, unsigned short port)
{
    sockaddr_in res;

    res.sin_family = AF_INET;
    res.sin_addr.s_addr = inet_addr(address);
    res.sin_port = htons(port);

    return res;
}

void connect_socket_with_address(SOCKET socket_descriptor, sockaddr_in
bind_addr)
{
    if (connect(socket_descriptor, (sockaddr *)&bind_addr, sizeof(bind_addr)) ==
SOCKET_ERROR)
        throw_error_with_code();
}

```

```

////////////////////////////////////

void disconnect(SOCKET connection)
{
    std::clog << "start disconnect..." << std::endl;

    if (closesocket(connection) == SOCKET_ERROR)
        throw_error_with_code();
    WSACleanup();

    std::clog << "disconnected" << std::endl;
}

////////////////////////////////////

SOCKET wait_client(SOCKET connection)
{
    sockaddr_in temp;
    int size_temp = sizeof(sockaddr_in);
    SOCKET client = accept(connection, (struct sockaddr *)&temp, &size_temp);

    if (client == INVALID_SOCKET)
        throw_error_with_code();

    return client;
}

////////////////////////////////////

struct UserInfo
{
    const char *login;
    const char *password;
};

void send(UserInfo user_info);
void get(UserInfo user_info);

int main(int argc, char *argv[])
{
    UserInfo user_info = {0};
    if (argc == 3)
    {
        user_info.login = argv[1];
        user_info.password = argv[2];
    }
    else
        throw "2 argumnts not wrotten\n";

    int selected_value = 0;
    while (true)
    {
        cout << "Change action:\n"

```

```

        << " 1. Send\n"
        << " 2. Get\n"
        << " 3. Exit"
        << endl;
    cin >> selected_value;

    switch (selected_value)
    {
    case 1:
        send(user_info);
        break;
    case 2:
        get(user_info);
        break;
    case 3:
        exit(0);
        break;
    default:
        cout << "Wrong input" << endl;
    }
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

return 0;
}

void input_message(string &dst, UserInfo user_info)
{
    string full_message;
    string line;
    while (getline(cin, line) && line != ".")
    {
        full_message += line + "\r\n";
    }

    dst = "From: smtp-client <" + string(user_info.login) + ">\r\n";
    dst += "Subject: smtp-client testmsg\r\n";
    dst += full_message + "\r\n.\r\n";
}

void send(UserInfo user_info)
{
    startup_wsa();

    SOCKET sock = get_connect(getaddrbyname("smtp.freemtpservers.com"),
SMTP_PORT);
    char cmd_buf[STR_BUF_SIZE];
    char text_buf[TEXT_BUF_SIZE];

    // блок конверта
    recv(sock, text_buf, TEXT_BUF_SIZE, 0);
    printf("Server on start: %s", text_buf);
}

```

```

    sprintf(cmd_buf, "HELO freesmtpservers.com\r\n");
    send(sock, cmd_buf, strlen(cmd_buf), 0);
    memset(text_buf, '\0', TEXT_BUF_SIZE);
    recv(sock, text_buf, TEXT_BUF_SIZE, 0);
    printf("Server on hello: %s", text_buf);

    sprintf(cmd_buf, "MAIL FROM: <%s>\r\n", user_info.login);
    send(sock, cmd_buf, strlen(cmd_buf), 0);
    memset(text_buf, '\0', TEXT_BUF_SIZE);
    recv(sock, text_buf, TEXT_BUF_SIZE, 0);
    printf("Server on mail: %s", text_buf);

    cout << "Input recipient: ";
    cin >> text_buf;
    sprintf(cmd_buf, "RCPT TO: <%s>\r\n", text_buf);
    send(sock, cmd_buf, strlen(cmd_buf), 0);
    memset(text_buf, '\0', TEXT_BUF_SIZE);
    recv(sock, text_buf, TEXT_BUF_SIZE, 0);
    printf("Server on recipient: %s", text_buf);

    send(sock, "DATA\r\n", 6, 0);
    memset(text_buf, '\0', TEXT_BUF_SIZE);
    recv(sock, text_buf, TEXT_BUF_SIZE, 0);
    printf("Server on data: %s", text_buf);

    // блок сообщения
    cout << "Input message:" << endl;
    string message;
    input_message(message, user_info);
    send(sock, message.c_str(), message.length(), 0);

    printf("\n\n--- msg ---\n\n%s\n\n", message.c_str());

    memset(text_buf, '\0', TEXT_BUF_SIZE);
    recv(sock, text_buf, TEXT_BUF_SIZE, 0);
    printf("Server on message: %s", text_buf);

    // завершение сессии
    send(sock, "QUIT\r\n", 6, 0);
    memset(text_buf, '\0', TEXT_BUF_SIZE);
    recv(sock, text_buf, TEXT_BUF_SIZE, 0);
    printf("Server on quit: %s", text_buf);

    disconnect(sock);
}

void read_resp(SOCKET sock)
{
    char buffer[TEXT_BUF_SIZE];
    int bytes;
    int total_bytes = 0;

```

```

    while ((bytes = recv(sock, buffer + total_bytes, TEXT_BUF_SIZE - total_bytes
- 1, 0)) > 0)
    {
        total_bytes += bytes;
        buffer[total_bytes] = '\0';

        // Проверяем завершение ответа (для однострочных команд)
        if (strstr(buffer, "\r\n"))
        {
            printf("%s", buffer);
            break;
        }

        // Проверяем завершение многострочного ответа
        if (strstr(buffer, "\r\n.\r\n"))
        {
            printf("%s", buffer);
            break;
        }

        // Защита от переполнения буфера
        if (total_bytes >= TEXT_BUF_SIZE - 1)
        {
            buffer[TEXT_BUF_SIZE - 1] = '\0';
            printf("%s", buffer);
            break;
        }
    }

    if (bytes <= 0)
    {
        printf("Connection closed or error\n");
    }
}

void send_cmd(SOCKET sock, const char *cmd)
{
    send(sock, cmd, strlen(cmd), 0);
}

void get(UserInfo user_info)
{
    startup_wsa();

    SOCKET sock = get_connect(getaddrbyname("pop.mypop3.com"), POP3_PORT);
    char cmd_buf[STR_BUF_SIZE];
    char input_buf[STR_BUF_SIZE];
    int message_count = 0;

    printf("Server on start: ");
    read_resp(sock);

    // авторизация

```

```

    sprintf(cmd_buf, "USER %s\r\n", user_info.login);
    send_cmd(sock, cmd_buf);
    printf("Server on user: ");
    read_resp(sock);

    sprintf(cmd_buf, "PASS %s\r\n", user_info.password);
    send_cmd(sock, cmd_buf);
    printf("Server on password: ");
    read_resp(sock);

    // запрос количества писем
    send_cmd(sock, "STAT\r\n");
    printf("Server on stat: ");
    char* stat_response = read_resp(sock);
    if (stat_response != NULL) {
        sscanf(stat_response, "+OK %d", &message_count);
        printf("Total messages: %d\n", message_count);
    }

    // запрос списка писем
    send_cmd(sock, "LIST\r\n");
    printf("Server on list: ");
    read_resp(sock);

    printf("Enter commands ('see N' to view or 'del N' to delete, 'quit' to
exit):\n");
    while (1) {
        printf("> ");
        fgets(input_buf, STR_BUF_SIZE, stdin);
        input_buf[strcspn(input_buf, "\n")] = '\0';

        if (strncmp(input_buf, "see ", 4) == 0) {
            int msg_num = atoi(input_buf + 4);
            if (msg_num > 0 && msg_num <= message_count) {
                sprintf(cmd_buf, "RETR %d\r\n", msg_num);
                send_cmd(sock, cmd_buf);
                printf("Server on retr %d: ", msg_num);
                read_resp(sock);
            } else {
                printf("Invalid message number. Available: 1-%d\n",
message_count);
            }
        }
        else if (strncmp(input_buf, "del ", 4) == 0) {
            int msg_num = atoi(input_buf + 4);
            if (msg_num > 0 && msg_num <= message_count) {
                sprintf(cmd_buf, "DELE %d\r\n", msg_num);
                send_cmd(sock, cmd_buf);
                printf("Server on dele %d: ", msg_num);
                read_resp(sock);
            } else {
                printf("Invalid message number. Available: 1-%d\n",
message_count);
            }
        }
    }

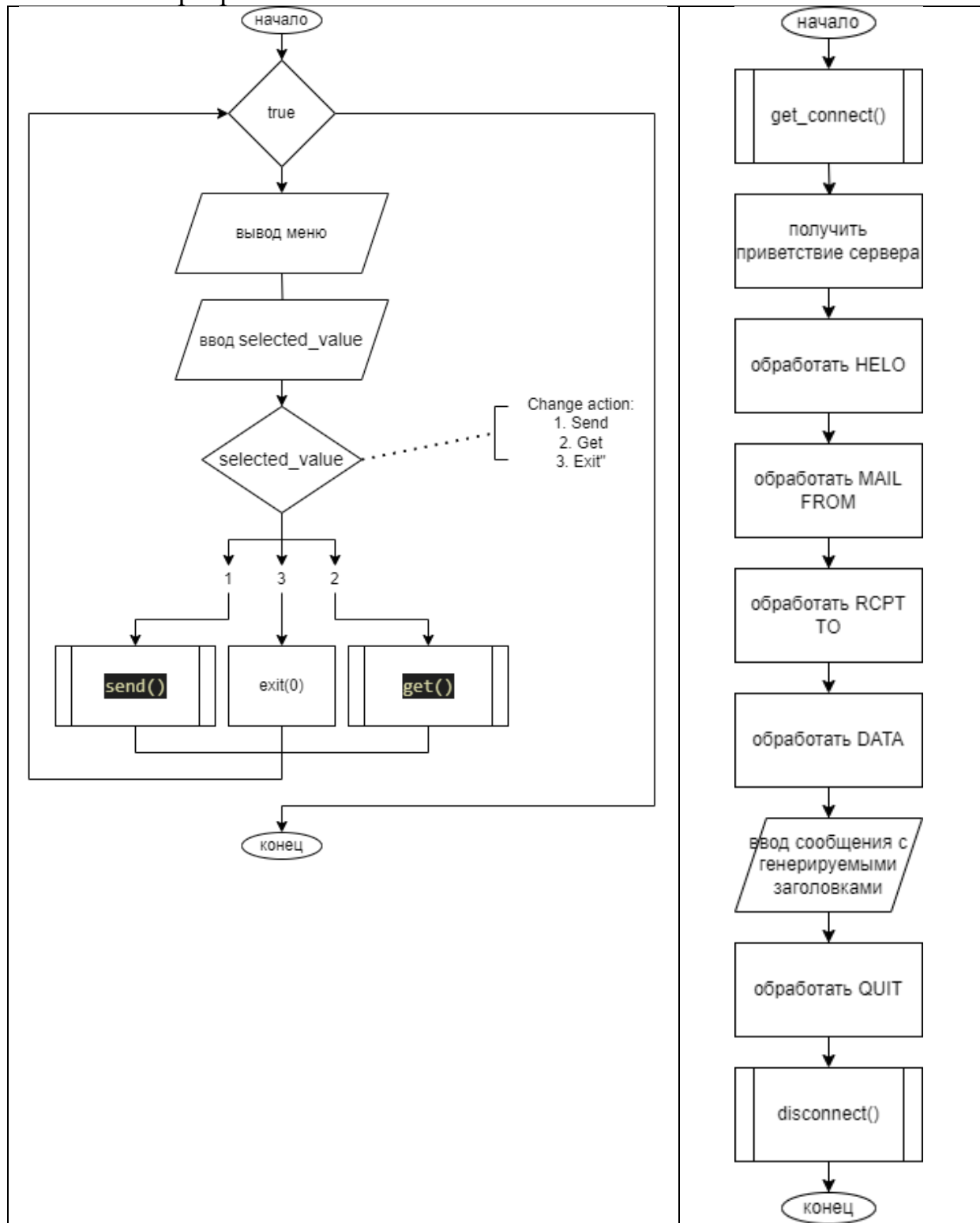
```

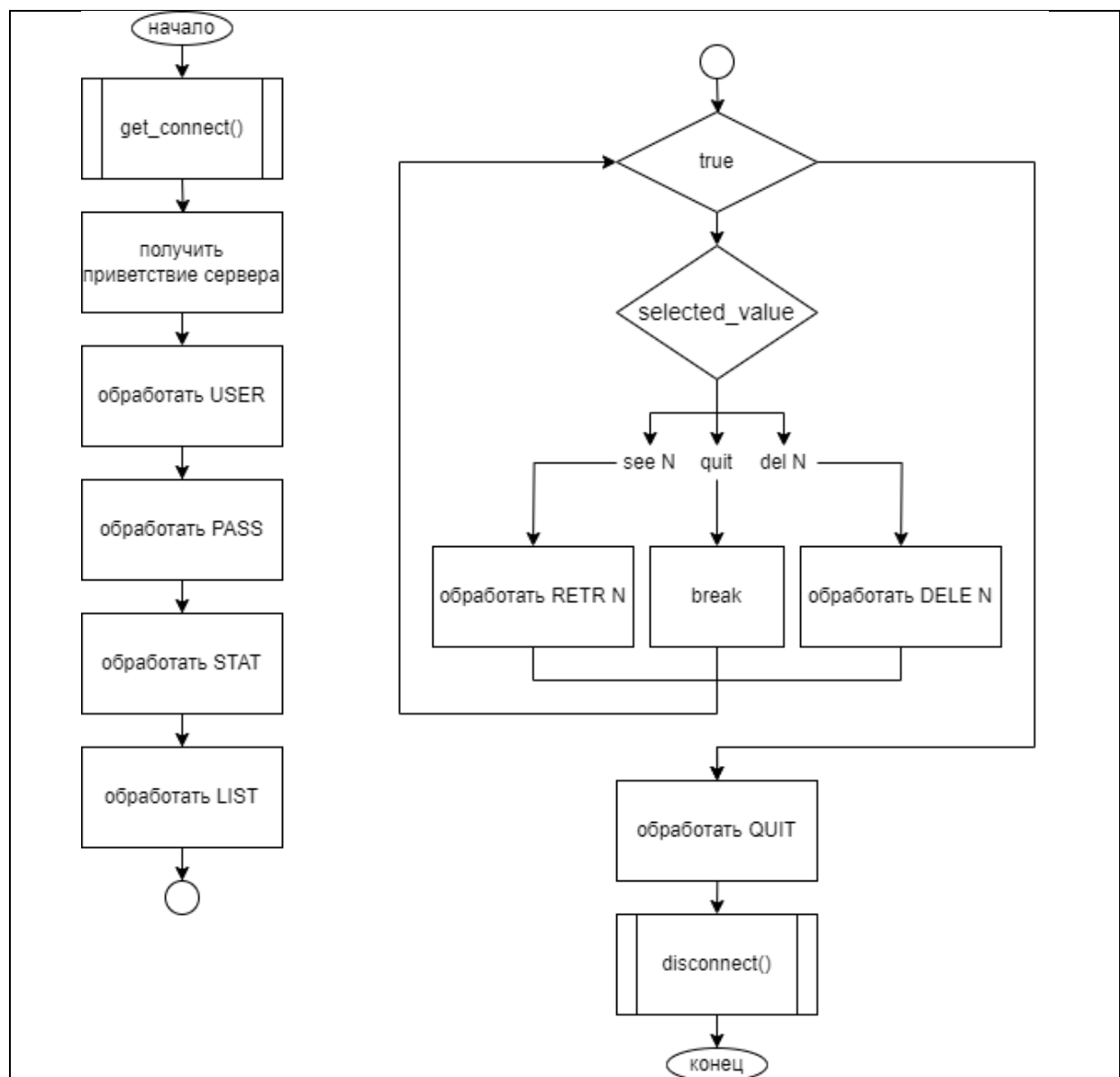
```
        }
    }
    else if (strcmp(input_buf, "quit") == 0) {
        break;
    }
    else {
        printf("Unknown command. Use 'see N', 'del N' or 'quit'\n");
    }
}

// завершение сессии
send_cmd(sock, "QUIT\r\n");
printf("Server on quit: ");
read_resp(sock);

disconnect(sock);
}
```

Блок схемы программ:





Анализ функционирования разработанных программ

SMTP:

```
C:\Users\dmitr\Projects\BSTU_lab\3kurs2sem\КомпCети\lab7>main i.am@example.com 1234
Change action:
  1. Send
  2. Get
  3. Exit
1
start connect...
WSA started...
create socket
create bind address
connect socket with address
connected
Server on start: 220 tools.wpoven.com Python SMTP 1.4.2
Server on hello: 250 tools.wpoven.com
Server on mail: 250 OK
Input recipient: dmitriykubik@gmail.com
Server on recipient: 250 OK
Server on data: 354 End data with <CR><LF>.<CR><LF>
Input message:
hello i am andrew as u. Please ask on one question. Why u do this? please, ask. dont ignore. </br> Andrew
Its not end, u know.
I can write on another line.
My body on the line, understood.
.

--- msg ---

From: smtp-client <i.am@example.com>
Subject: smtp-client testmsg

hello i am andrew as u. Please ask on one question. Why u do this? please, ask. dont ignore. </br> Andrew
Its not end, u know.
I can write on another line.
My body on the line, understood.
.

Server on message: 250 OK
Server on quit: 221 Bye
start disconnect...
disconnected
```

Subject : [smtp-client testmsg](#)

From : i.am@example.com To : dmitriykubik@gmail.com

Dated : 2025-05-10 10:18:40 ([Delete](#))

[HTML](#)

[HTML Source](#)

[Text](#)

[RAW](#)

hello i am andrew as u. Please ask on one question. Why u do this? please, ask. dont ignore. </br>
Andrew
Its not end, u know.
I can write on another line.
My body on the line, understood.

В роли сервера SMTP использовался сервис wpoven.com, позволяющий производить передачу через порт 25.

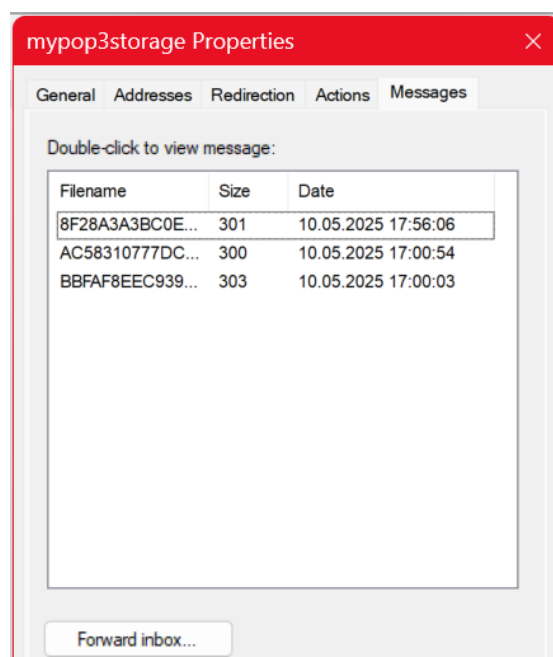
POP3:

```
C:\Users\dmitr\Projects\BSTU_lab\3kurs2sem\КомпСети\lab7>main m@mypop3.com m
Change action:
  1. Send
  2. Get
  3. Exit
2
start connect...
WSA started...
create socket
create bind address
connect socket with address
connected
Server on start: +OK POP3 server ready
Server on user: +OK
Server on password: +OK
Server on stat: +OK 3 904
Total messages: 3
Server on list: +OK
1 303
2 300
3 301
.

Enter commands ('see N' to view or 'del N' to delete, 'quit' to exit):
> see 1
Server on retr 1: +OK
From: smtp-client <user@mypop3.com>
Subject: smtp-client testmsg

test test

> del 1
Server on dele 1: +OK Message marked for deletion
> quit
Server on quit: +OK Goodbye
start disconnect...
disconnected
```



В роли сервера POP3 использовалось приложение MailEnable с собственным сервером с доменом турор3.com.

Вывод: в ходе выполнения лабораторной работы были изучены принципы работы протоколов SMTP и POP3. SMTP – это протокол, который поддерживает только отправку сообщений электронной почты. POP3 – протокол, который поддерживает получение сообщений. Данные протоколы используют протокол TCP/IP как транспортное средство для передачи сообщений.